Cliente HTTP síncrono

Elaborada por: M. en C. Ukranio Coronilla

Vamos a analizar el funcionamiento de un cliente HTTP moderno de Java (Java 11+) el cual se encuentra disponible en la sección ***"1. Synchronous Example"*** del siguiente enlace:

https://mkyong.com/java/java-11-httpclient-examples/

Compílelo y ejecútelo para verificar su funcionamiento.

Podemos en este código observar que tanto para crear un objeto del tipo HttpClient como para crear un objeto del tipo HttpRequest en lugar de usar new junto con un constructor público con n parámetros, se utiliza un **patrón Builder**. Esta forma de crear objetos es muy usada en clases abstractas donde el diseñador quiere tener control total sobre cómo se crea el objeto. Si existiera un constructor público como:

```
new HttpClient(null, null, null, 0, false, null);
```

se tienen que dar todos los valores de los parámetros forzosamente. Al ser varios parámetros puede haber errores de posición y/o valor, y lo más importante, al haber cambios se requerirá de un nuevo constructor. Observe en el código que el método `build()` es el que crea el objeto con los valores dados a los parámetros que se deseen.

La clase `HttpClient` se usa para construir un cliente HTTP. En este caso el objeto `httpClient` se configura para usar la versión HTTP 1.1 y en caso de no conectarse después de 10 segundos lanza una excepción.

La clase `HttpRequest` se usa para construir la petición HTTP. En este caso el objeto `request` se configura para ser un método HTTP GET, y se especifica como URL del servidor la siguiente https://httpbin.org/get que es un servicio de "espejo" (echo) diseñado para pruebas. Su función principal es devolver un objeto JSON en el campo de datos del mensaje HTTP, que contiene todos los detalles de la petición que se le

envíe, como son los parámetros, headers, IP pública y URL. Finalmente se agrega el header personalizado `User-Agent`.

Posteriormente el método `send` realiza el envío síncrono del mensaje HTTP (almacenado en el objeto `request`), por lo que el hilo se bloquea hasta que se reciba una respuesta o se agote el timeout de 10 segundos:

```
HttpResponse<String> response =
httpClient.send(request, HttpResponse.BodyHandlers.ofString());
```

En el segundo parámetro tenemos un "traductor" que convierte los bytes recibidos en el cuerpo del mensaje a un String. Por último, el objeto `response` es un objeto genérico inmutable de la interfaz `HttpResponse`. El cual almacena el cuerpo del mensaje, el código de estado y los encabezados de respuesta.

Esta instrucción requiere manejar las excepciones (`IOException` e `InterruptedException`), ya que la red puede fallar o la conexión puede cortarse inesperadamente. Por esa razón en lugar de usar `try-catch` se declaran con `throws` en la función principal.

En los siguientes ejercicios intente programarlos preguntando a la IA lo necesario para aprender sobre streams y expresiones Lambda en Java. Envíe código fuente y capturas de pantalla como comprobantes de la presente práctica.

Ejercicio 1

Pida a la IA que le explique la línea de código:

```
HttpHeaders headers = response.headers();
```

Posteriormente substituya la línea:

```
headers.map().forEach((k, v) -> System.out.println(k + ":" + v));
```

por código que igualmente imprima los headers pero accediendo a los elementos individuales del mapa `Map<String,List<String>>` con ayuda de un iterador.

Ejercicio 2

Retome el código del ejercicio anterior y copie el mapa `headers` recibido en un nuevo mapa, y posteriormente agregue los siguientes headers al nuevo mapa:

```
Set-Cookie: Max-Age=0
Set-Cookie: id=123
Set-Cookie: theme=dark
```

imprima el contenido del nuevo mapa (mantenga también la impresión del ejercicio 1) usando el método `forEach` (Pregunte a la IA sobre los casos en que puede usarse y cómo se usa).

Ejercicio 3

Retome el código del ejercicio anterior y utilice además de `forEach` a los métodos `entrySet` y `sorted` de la interfaz Stream para imprimir los headers ordenados alfabéticamente por su nombre (llave).